vcebott Question Bank - Build Plan

Status: plan, awaiting confirmation. Date: 2026-10-04.

1. Goal

A browser where VCE Methods and Specialist students find past VCAA exam questions filtered by topic and difficulty. Difficulty comes from state-wide mark distributions in external assessment reports.

2. Confirmed decisions

Decision	Choice
Difficulty basis	Raw proportion plus bank-normalized percentile; filter on proportion, percentile toggle
Question display	PDF crops rendered as images (faithful math and figures)
Year coverage	All years 2006-2026, NHT sessions excluded (no published statistics)
Subject order	Methods first, then Specialist port
OCR engine	Gemini 2.5 Flash-Lite API (paid, approximately USD 0.1-0.4 one-time)
Topic classification	Bake-off: Flash-Lite multi-label vs Simple Jev threshold, decided by manual verification on a shared test set
Classification granularity	Whole question only, never per part (parts appear as statistics in the detail view)
Question storage	PNG crop files plus one bank parquet per subject in `data/processed/`
Text search	Basic substring/FTS search over OCR text in v1

Hardware basis: no GPU, 15 GB RAM, 12 cores. This rules out a local VLM and confirms the API choice; Jev remains viable on CPU.

3. Existing pipeline (verified)

```
scrape  ->  parse  ->  analyze  ->  classify  ->  serve
(done)     (done)     (done)       (not run)     (marks-entry dashboard)
```

Existing processed tables for methods:

- `marks`: per question, per mark level, percent of state achieving it (`subject, year, exam, nht, question, marks_available, mark, pct, average`).
- `separators`: `average, top_quartile_mean, gap, pct_zero, gap_rank` per question (gap = top quartile mean minus overall mean).
- `question-text`: pdftotext blocks, currently mangled (`fi` ligature becomes `TH`, superscripts and fractions destroyed). Unused by any route.
- `topics` and `topic-difficulty`: never generated (Jev server never run).

4. New pipeline

```
scrape -> parse -> crop -> ocr -> classify -> build-bank -> serve /bank
                      (new)  (new)  (existing)    (new)         (new route)
```

4.1 Crop (new module, PyMuPDF)

- For each paper PDF: locate `Question N` heading rectangles, crop from heading to next question heading or end marker, render PNG.
- Output: `data/processed/<subject>-question-images/<year>-<exam>[-nht]/<label>.png` plus an index parquet with the crop rectangle for cache keys.
- Old PDFs (2006-2012) have different heading formats; per-era detection rules, fallback to full-page image when heading detection fails.
- Multi-page questions: crop each page segment, store ordered list.

4.2 OCR (new module, Gemini Flash-Lite REST)

- One image per request, temperature 0, structured JSON response: `{"latex": "...", "plain": "...", "notes": "..."}`.
- Plain HTTP via httpx to the generativelanguage REST endpoint with response schema; no new SDK dependency.
- Cache keyed by `(file sha256, crop rect)`: interrupted runs resume free, re-runs never re-bill.
- NHT sessions skipped (no mark statistics exist for them).
- OCR purpose: clean searchable text and clean classifier input. Display stays crop images; KaTeX client-side render from `latex` is a secondary view (CDN script, zero cost).

4.3 Classify (bake-off between two candidates)

Topic vocabulary comes from `Topics.csv` at repo root and replaces the built-in 8-label list:

- Methods (4): Functions, relations and graphs; Algebra; Calculus; Probability and Statistics.
- Specialist (8): Proofs and Logic; Counting; Probability and Statistics; Trigonometry; Linear Algebra; Algebra over C; Functions, relations and

graphs; Calculus.

Questions may carry multiple topics (multi-label). Classification runs per whole question block, never per part.

Candidates run on the same test set (existing `tests/fixtures/ topic-sample.jsonl`, remapped to the new vocabulary), then every question where the two candidates disagree is listed for manual verification. The candidate that is correct more often wins; the losing path is deleted.

- **Candidate B - Flash-Lite multi-label**: adds a `topics` array to the existing OCR JSON schema, so one call returns latex, plain text, and topic labels with confidence. Approximately USD 0.05 extra for the full bank. No Jev server dependency.
- **Candidate C - Simple Jev threshold**: one Jev call per question returns a probability for every topic; every topic above a swept threshold becomes a label. Threshold is chosen from the same evaluation (sweep 0.15, 0.20, 0.25, 0.30, 0.40). Runs on CPU, free.

Fixture remapping note: the old single-label vocabulary collapses into the new coarser one (`algebra-functions` maps to Functions, relations and graphs; both calculus labels map to Calculus; trigonometry, statistics and both probability labels fold into the nearest new bucket). Ambiguous remapped samples are re-verified during the manual pass, which also grows the test set.

After the winner is chosen: tags whose confidence falls below the winning configuration's threshold are flagged for review in the UI rather than silently trusted.

4.4 Build bank (new module)

One row per question block, merged from marks + separators + OCR + topics:

Column	Source	Note
subject, year, exam, nht	session identity	
label	report numbering	`1`, `2b`
marks	marks table	
avg_proportion	`average / marks_available`	primary difficulty metric, 0-1 scale
difficulty_percentile	rank of avg_proportion across bank	normalized slider view
pct_zero	separators	share of state receiving zero
gap	separators	discrimination, secondary filter
topic	Jev or Flash-Lite winner	list of labels per question (multi-label)

Column	Source	Note
topic_confidence	same	list, aligned with topic list
image_paths	crop index	ordered list, first page segment first
latex_text, plain_text	OCR	search and KaTeX view

CLI: `uv run vcebott build-bank methods`.

Granularity rule: report labels are part-level (`2a`), paper text blocks are whole-question (`2`). Bank normalizes to whole-question; part statistics are aggregated weighted by marks and kept for the detail view.

Difficulty interpretation note for UI: multiple-choice questions (Exam 2 Section A) are binary, so avg_proportion equals percent correct there; extended-response values are graded. Same 0-1 scale, meaning noted on hover.

4.5 Serve /bank (new route + template)

- Filters: subject, topic (multi), difficulty range (slider on proportion, toggle to percentile), marks, exam 1/2, year range, separators only, NHT off by default, text search box.
- Sort: difficulty, year, or gap.
- Card: crop image, difficulty badge (proportion plus percentile), topic with confidence, marks, link to paper PDF and assessment report.
- Detail: part-level mark distributions, KaTeX alternate render, state distribution table.
- Existing routes untouched.

4.6 Storage

Question content lives in layers, all under `data/` (gitignored, reproducible via CLI):

1. Raw provenance (existing): papers and reports in `data/raw/<subject>/<year>-exam<N>[-nht]/`; source URLs and sha256 hashes in `data/manifest.json`. Never modified.
2. Question images (new): `data/processed/<subject>-question-images/` with one PNG per page segment, paths `.../<year>-<exam>[-nht]/<label>.png`, ordered list per question. Files, not database blobs, because the dashboard serves them directly via FastAPI StaticFiles.
3. Bank (new): `data/processed/<subject>-bank.parquet`, one row per whole question with every column from section 4.4 plus a nested list column of part-level statistics. Search is pandas substring filtering over `plain_text`; roughly 1600 rows, no index needed in v1.
4. Resumable caches (new): `<subject>-ocr-cache.parquet` and `<subject>-classify-cache.parquet`, keyed by file sha256 plus

crop rectangle, so interrupted or repeated stages never redo paid or slow work.

Parquet over SQLite: matches the repo convention of one parquet per stage, zero new infrastructure, small row counts. SQLite FTS stays a later option if substring search proves too coarse.

`<subject>-question-text.parquet` survives only as the raw pdftotext record; the bank and classifier consume OCR text from 4.2 onward. Either classification winner runs on OCR text, so the mangled-text classification path is removed at cutover.

5. Phases and acceptance

1. **Crops** - render.py, per-era heading rules. Accept: visual sample of 30 crops spanning 2006-2026 all cleanly bounded.
2. **OCR trial** - 20-question batch through Flash-Lite including the B candidate `topics` array. Accept: LaTeX and labels spot-check pass, measured cost within estimate, then full methods run.
3. **Classify bake-off** - remap fixture to `Topics.csv` vocabulary, run candidates B and C on the same labeled set, sweep C thresholds, export every disagreement to a review CSV. Accept: user manually verifies the disagreement list and picks the winner; losing path deleted at cutover.
4. **Build bank + /bank UI** - build-bank merge with winning classifier output; methods UI. Accept: every filter combination returns consistent rows, images render, detail view correct.
5. **Specialist port** - extended vocabulary, layout verification. Accept: same checks as phases 1-4 on Specialist.
6. **Docs and cleanup** - README update, throwaway scripts removed.

6. Risks

- Crop boundary detection across 20 years of layouts: mitigate with per-era rules and page-image fallback; sample verification per decade.
- Flash-Lite LaTeX quality on dense set notation: trial batch gates the full run; Flash upgrade costs about USD 1 total if needed.
- 0.8B Jev reliability on tail labels in multi-label mode: bake-off decides empirically; losing candidate removed; manual review path stays open.
- Part/block label mismatch: weighted aggregation rule above; row-count reconciliation in phase 3.
- Old-era statistics differ in granularity (percent achieving full marks only): bank records what each era publishes; UI marks sparse eras.

7. Prerequisites

- Gemini API key (environment variable, not committed).
- New dependency: pymupdf. No other new runtime dependencies.

- Simple Jev server needed only if candidate C wins the bake-off (README launch command already exists).